COMS BC3997 (Applied Cryptography) October 21, 2026

Homework 2: Anonymous Voting

Instructor: Eysa Lee *Due: November 6, 2026*

In this assignment, you will implement a simple cryptographic voting protocol using homomorphic encryption and zero-knowledge proofs. This system prevents an adversary from modifying votes, voting more than once, or learning an individual's vote (beyond what can be inferred from the final outcome).

1 AI, External Resources, and Collaboration

AI Usage. You are permitted the use of AI to assist in syntax and debugging your program assignments. However, you are *not* allowed to have AI code up the entire assignment for you. You are also not allowed to copy code from libraries, implementations, or elsewhere. You should, using your knowledge from class and course materials, come up with the design of your program without the assistance of AI or other external resources. Violation of this policy is considered an honor code violation and may result in a failing grade for the assignment (and possibly the entire course).

Collaborations. Collaborating with classmates is encouraged and allowed. Code walks are individual, and each student will be expected to fully understand and be prepared to talk about their submissions on their own. Collaborations should be acknowledged in the design document submitted alongside the code.

2 Assignment Specification

In this assignment, you will be implementing a simplified version of the cryptographic voting protocol Helios. There will be a **Registrar** that checks voters are registered and only vote once, **Arbiters** that will generate the election parameters and decrypt the final tally, and multiple **Voters** that can vote, view, and verify the final result.

You are allowed and encouraged to reference Cryptographic Voting — A Gentle Introduction for details on the concepts we discussed in class. Please note the writeup may contains slight differences from what is required in this assignment. If you would like to implement anything beyond the base specifications of this assignment, please ensure these differences are noted in your design document. You must also include in what ways your changes improve upon the basic requirements.

2.1 Setting

Assumptions. For this assignment, you may assume existence of a Public Key Infrastructure (PKI) that handles distributing public keys for all parties. That is, you may directly give all parties the public keys for other parties.

You may also assume the existence of an anonymous broadcast channel, which will allow any party to anonymously send to all other parties any message. For the base version of your protocol, you may implement the anonymous broadcast channel by assuming the existence of an honest third party who will verify messages received from parties and honestly relay the same message to all other parties. *Note that this honest third party may ONLY be used in this way and will NOT collude with other parties.* For extra credit, improve upon this aspect of the protocol by removing the need for an honest third party. See extra credit section for more details.

Adversary. Throughout the protocol, **you must ensure that all communication is secure against an adversary that can intercept and possibly modify messages**. You may assume the Registrar will follow the protocol honestly but may collude with others to try to learn extra information. You may assume that there is always at least one honest voter and one honest arbiter who will act honestly and not attempt to collude with others. However, you do not know before the end of the protocol which voter(s) and arbiter(s) are honest. During the protocol, dishonest voter(s) and arbiter(s) may possibly try to deviate from the protocol or collude with others to learn extra information. **You are expected to catch voter(s) and/or arbiter(s) who attempt to cheat and ignore the cheating/malformed contribution**.

2.2 Roles

Registrar. The purpose of the registrar is to act as a central server that maintains a database of parties that are allowed to vote and ensures that (1) only registered parties may cast a vote and (2) each registered party only casts at most one vote. For simplicity, the IDs of each voter will be a unique number 1 through n (where n is the number of voters), and the registrar will be given n.

Whenever a voter is ready to cast their vote, they will communicate with the registrar to receive a signature on an encryption of their vote. The registrar should check that the voter is registered and that the voter has not previously received a signature on a vote. If both conditions hold, the registrar will issue a Blind RSA signature on an encryption of the vote. (For simplicity, voters may not change their vote.)

Arbiters. The purpose of the arbiters is to jointly generate El-Gamal encryption and decryption keys. Arbiters will jointly sample a random El-Gamal secret key by each (locally) sampling their own random value from $\mathbb{Z}_q$. Each arbiter will define the secret key to be the sum of their local random values (i.e., if there are t arbiters and each arbiter holds the value $x_i \in \mathbb{Z}_q$, then the El-Gamal secret key is defined as

$x = \sum_{i=1}^{t} x_i \mod q$)[1]. To collectively compute the corresponding public key g^x, each arbiter will publish their share of the public key g^{x_i} along with a zero-knowledge proof of knowledge of discrete log for g^{x_i}.

Once the voting phase has closed, arbiters will use the homomorphic properties of El-Gamal to compute the encrypted tally from the votes that were broadcast. Each arbiter will use their x_i to compute a partial decryption of the encrypted tally and publish the partial decryption along with a zero knowledge proof that the partial decryption, g^{x_i}, and the first part of the encrypted tally ciphertext are a DDH tuple.

Voters. Voters will use the El-Gamal public key published by the arbiters to encrypt a value of 0 or 1. To register their vote, they will have the registrar issue a Blind RSA signature on the encryption of the vote. Voters will anonymously broadcast their encrypted votes, their unblinded signatures, and zero-knowledge proofs that each of their ciphertexts correspond to zero or one.

Once the voting phase has closed, voters can compute the encrypted tally and (attempt to) decrypt the final result using the partial decryptions from the arbiters.

3 Main Program API Specification

The following detail the commands your program should handle and expected behavior. Parameters will be input in hex, and all other values will be input as strings. During testing and grading, we will not input any non-standard or weird character codes.

setup_params

Load the public parameters for El-Gamal (modulus p, group generator g, and group order q) from a file titled `parameters`. Values p, g and q will be given in `parameters` in hex on separate lines.

init_parties [t] [n]

This should initiate t arbiters, n voters, and the registrar. Each voter should be assigned a unique identifier between 1 and n. Parties should be started up separately from each other and sample the public key(s) they will use for communication. All parties should communicate their public keys to each other. We will assume in this stage the public keys sent and received are not tampered with.

elgamal_keygen

Arbiters should jointly run a distributed El-Gamal Key Generation. In particular, each arbiter should sample a their random share of the secret key x_i, and send to all parties the value g^{x_i} along with a zero-knowledge proof of knowledge of discrete log for g^{x_i}. All parties should compute the joint public key $g^x = \Pi_{i=1}^{t} g^{x_i}$. Output onto standard out the value of the public key g^x. This command will only be run once.

register_vote [ID] [vote]

This takes in two numbers corresponding to the ID number of the voter and their vote. The voter will use the public key g^x (produced by elgamal_keygen) to encrypt its vote. The voter will then initiate the protocol to receive a Blind RSA signature from the registrar on its encrypted vote $c = (c_1, c_2)$. First, the voter will sample a random "masking" element $r \leftarrow \mathbb{Z}_N^*$ compute its masked message as $m' = H(m) \cdot r^e$, and send the masked message to the registrar. The registrar will compute the blind signature $\sigma' = (m')^d$ and send this back to the voter. The voter can then unmask the signature by computing $\sigma = (\sigma') \cdot r^{-1}$.

[1] Arbiters may use their share x_i in subsequent computation, but at no point should an arbiter publish their share directly. Do not have arbiters send each other their share of the secret key to directly compute the secret key!!

cast_votes

Voters will use the anonymous broadcast channel to broadcast the encryption of their vote, the (unblinded) signature they received from the registrar, and a zero knowledge proof that their ciphertext corresponds to an encryption of 0 or 1.

decrypt_tally

Arbiters will aggregate the encrypted votes sent via the anonymous broadcast channel to compute an encryption of the sum of the votes. In particular, if each ciphertext is of the form $c_i = (c_{1,i}, c_{2,i})$, then the encrypted tally is computed as $c = (c_1, c_2)$ where $c_1 = \Pi_{i=1}^{n} c_{1,i}$ and $c_2 = \Pi_{i=1}^{n} c_{2,i}$. Each arbiter will compute a partial decryption of the ciphertext as $d_j = c_1^{x_j}$ using its share of the secret key x_j. Arbiters will send to all parties their partial decryption d_j along with a proof that c_1, g^{x_j}, d_j are a DDH tuple. Once parties receive all t partial decryptions, they can compute the decryption from $g^m = c_2/(\Pi_{j=1}^{t} d_j)$. Output onto standard out the final tally of votes.

3.1 External Libraries

Allowed. You are also allowed to use libraries for basic group operations (e.g., exponentiation, modular arithmetic, computing inverses) and sampling randomness. Since secure messaging was covered in the previous homework, you may use existing implementations establishing secure communications between two parties (e.g., DH key exchange, authenticated encryption). You may also use existing implementations for secure hash functions.

Not Allowed. You may be use existing implementations for sampling random primes, but you are expected to choose (and compute) the correct parameters p, q (both for RSA and El-Gamal) yourself. You should not use existing implementations of the zero-knowledge proofs, blind RSA, distributed/threshold El-Gamal, or other topics we covered/introduced for this particular project. When in doubt, you should ask the instructor.

Suggested Libraries. You are not required to use the following libraries, but they contain building blocks that may be helpful for your assignment.

- C++: CryptoPP orAWS libcrypto
- Rust: Any of the libraries withing Rust Crypto

4 Submission

Your assignment should be submitted through Gradescope as a ZIP. You will submit the following four components:

1. Source code for your implementation (written in either Rust or C++).
2. A Makefile that compiles your implementation upon running `make`.
3. A main program and file titled "`COMMAND`" that contains on one line the command to run your main program.
4. A PDF titled "design" that includes your name, UNI, **any dependencies**, instructions for how to run your implementation by hand, and information about your implementation. You should include major design decisions, an outline of your code, and any other information you think is relevant to the grading of your assignment. If applicable, please also acknowledge any collaborators you worked with and if/how AI was used in your assignment.

Testing and Grading. For testing and grading, you will also submit a "main" program that can be used to interface with your implementation. Your "main" program may be in any language, but the your implementation for this assignment must be in Rust or C++. (It will likely be easiest to have your main program written in the same language as your implementation.)

The main program will read a sequence of newline delineated commands from standard input ending with EOF. When instructed to, it will display output to standard out. We will only use the main program in the way specified in the API specification and with valid inputs. We are not looking to trip you up on API edge cases.

Codewalks. There will be random 10-15 minute codewalks throughout the semester where you will be expected to explain not only your code but also design decisions. Codewalks are only announced after submissions close.

5 Extra Credit

For extra credit, discuss how we may remove the trusted third party we used for the anonymous broadcast channel. Note you may choose to address only achieving anonymity or only being a reliable broadcast. That is, we are guaranteed to hide the sender but a party who misbehaves could try to make parties to receive different messages. Or we can't hide the sender but we are guaranteed that every party will receive the same message in the end. In your design document, describe potential modifications, what you believe your potential modification to achieve, and what your potential modification does not achieve.